Task 4: Computer Vision Object Detector & Frame-by-Frame Segmenter Pipeline

Progree Remote Internship: Artificial Intelligence
Intern: **Darakhshan Mujtaba**

Model Performance Whitepaper

Abstract

This whitepaper presents a real-time computer vision pipeline, built with OpenCV, that segments moving objects in a video stream, classifies them by shape (circle, square, triangle) and overlays a live count for every class on each frame. The pipeline uses Gaussian filtering, adaptive thresholding, morphological cleanup and contour analysis. It was evaluated frame by frame on a 250-frame test video with known ground truth, uneven lighting and camera noise. The total object count was correct in **100.0%** of frames, compared with **6.8%** for a global-threshold baseline, and every class count was exactly right in **97.2%** of frames. Average inference time in Google Colab was **8.67 ms per frame (115 FPS)**, more than four times faster than the 25 FPS needed for real-time use.

1. Introduction

Counting and classifying objects in video is a basic building block of many vision systems, from counting parts on a conveyor belt to monitoring traffic. A key difficulty in real footage is uneven lighting: a single brightness cut-off that works on one side of the image fails on the other. The goal of this project was to build a pipeline that:

- processes a video stream frame by frame fast enough for real-time use,
- applies Gaussian filters and adaptive threshold matrices to separate objects from the background based on pixel values,
- extracts the contours of target objects and classifies them into shape classes,
- overlays dynamic text counts for each class onto every frame, and
- reports frame-by-frame inference metrics: accuracy against ground truth, latency and throughput.

2. System architecture

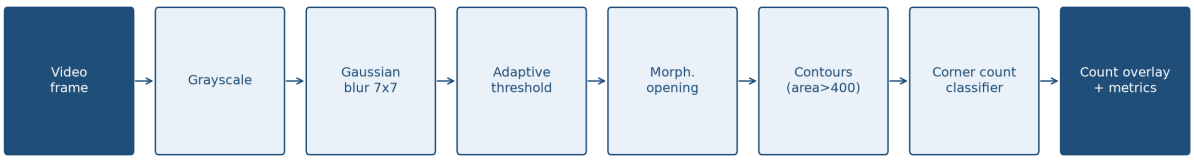


Figure 1. Processing pipeline applied to every video frame.

Stage	OpenCV operation	Purpose
Grayscale	cvtColor(BGR2GRAY)	Keep brightness only; reduce data by 3x
Gaussian blur	GaussianBlur, 7x7 kernel	Suppress pixel noise before thresholding
Adaptive threshold	adaptiveThreshold, Gaussian-weighted, 101x101 block, C = -15	Pixel is foreground if it is 15 levels brighter than its local neighbourhood
Morphological opening	morphologyEx(MORPH_OPEN), 5x5 kernel	Remove isolated white specks
Contour extraction	findContours(RETR_EXTERNAL), area > 400 px	One outline per object; ignore small blobs
Shape classification	approxPolyDP, epsilon = 4% of perimeter	3 vertices = triangle, 4 = square, more = circle
Overlay	drawContours, putText	Outline, label and live per-class count box

Table 1. Pipeline stages and parameters.

3. Methodology

3.1 Test video with ground truth

To measure accuracy, every frame needs a known correct answer. A 10-second test video (640 x 480 pixels, 25 FPS, 250 frames) was therefore generated programmatically. It contains 7 moving objects (circles, squares and triangles of radius 16 to 25 pixels) that enter and leave the scene at different times, so the number of objects changes from 0 to 7 during the video (1,143 object instances in total). Two realistic difficulties were added on purpose:

- **Uneven lighting:** the background brightness falls from 110 on the left to 20 on the right, and objects on the right are about 45% dimmer.
- **Camera noise:** Gaussian noise with a standard deviation of 12 grey levels is added to every pixel of every frame.

The true number of objects of each class in each frame was stored in a ground-truth file. Each object moves in its own horizontal lane so objects never overlap; occlusion is discussed as a limitation in Section 6.

3.2 Segmentation: Gaussian filter and adaptive threshold

After conversion to grayscale, a 7 x 7 Gaussian filter averages each pixel with its neighbours, weighting closer pixels more. This removes most of the grainy noise. The blurred frame is then turned into a binary mask. A **global** threshold (Otsu's method) chooses one cut-off value for the whole image. An **adaptive** threshold instead computes a separate cut-off for every pixel from a Gaussian-weighted 101 x 101 neighbourhood, so it follows the lighting across the frame.

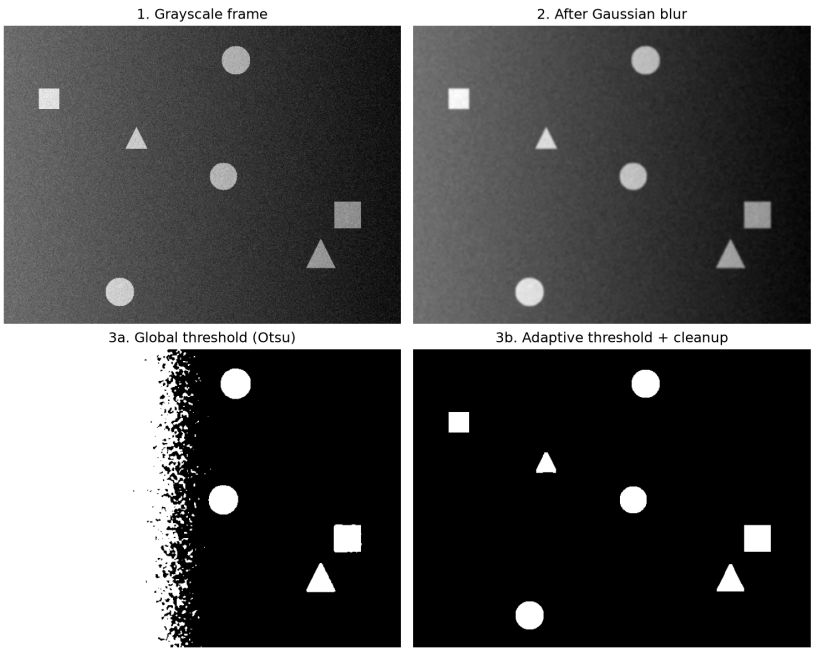


Figure 2. Frame 120 at each stage. The global threshold (3a) turns the bright left side white and loses objects; the adaptive threshold (3b) segments all 7 objects cleanly.

3.3 Contour extraction and classification

External contours are traced around each white region of the mask, and regions smaller than 400 pixels are discarded. Each contour is simplified with the Douglas-Peucker algorithm (approxPolyDP) using a tolerance of 4% of its perimeter. The number of remaining vertices decides the class: 3 for a triangle, 4 for a square, and more for a circle. Each object is then outlined and labelled in its class colour, and a count box in the top-left corner shows the live number of objects of each class (Figure 3).

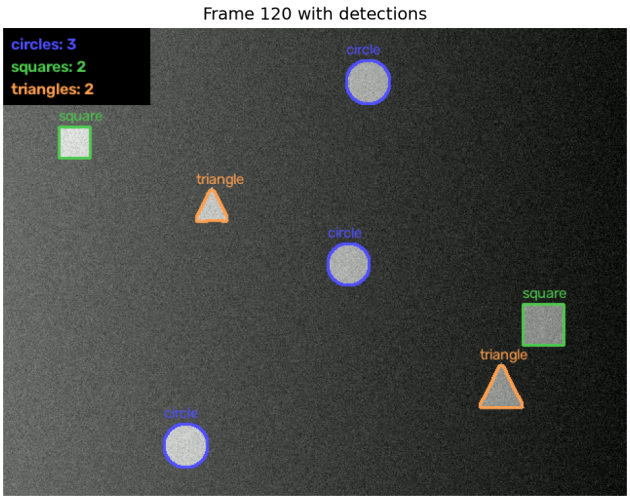


Figure 3. Pipeline output for frame 120: outlines, labels and the dynamic count overlay.

4. Evaluation methodology

The pipeline was run on every frame of the test video in Google Colab. For each frame the following were logged to frame_metrics.csv:

- **Inference time** (ms) for the complete pipeline, measured with time.perf_counter(), and the resulting throughput in frames per second.
- **Predicted vs. true count** for every class, giving the per-class count accuracy (share of frames with the exact right count) and mean absolute error (MAE).
- **Total object count** for both the adaptive pipeline and a global-threshold baseline that is identical except for the thresholding step.
- **Exact-frame accuracy**: the share of frames in which all three class counts are simultaneously correct.

5. Results

5.1 Detection and classification accuracy

Metric	Circle	Square	Triangle	Total
Frames with correct count	100.0%	97.2%	97.2%	100.0%
Mean absolute count error	0.000	0.028	0.028	0.000

Table 2. Per-class results of the adaptive pipeline over all 250 frames.

Method	Total count correct	Mean abs. error (objects)
Global threshold (Otsu) baseline	6.8% of frames	1.83
Adaptive threshold pipeline	**100.0% of frames**	**0.00**

Table 3. Effect of the thresholding method on object detection.

The adaptive pipeline detected every object in every frame: the total count was never wrong. All three class counts were exactly right in 97.2% of frames, and at object level 99.4% of the 1,143 object instances were classified correctly. The global threshold baseline counted correctly in only 6.8% of frames, because it merged the bright left part of the background into one large region and missed the objects inside it (Figure 4).

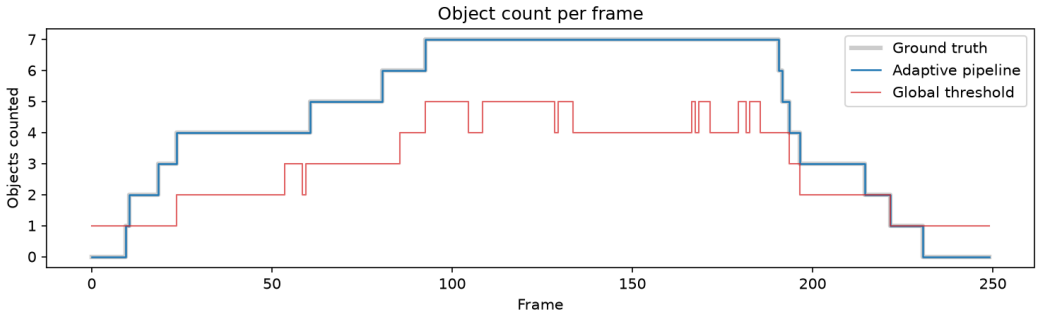


Figure 4. Objects counted per frame: ground truth, adaptive pipeline (overlapping the truth) and global-threshold baseline.

5.2 Inference speed

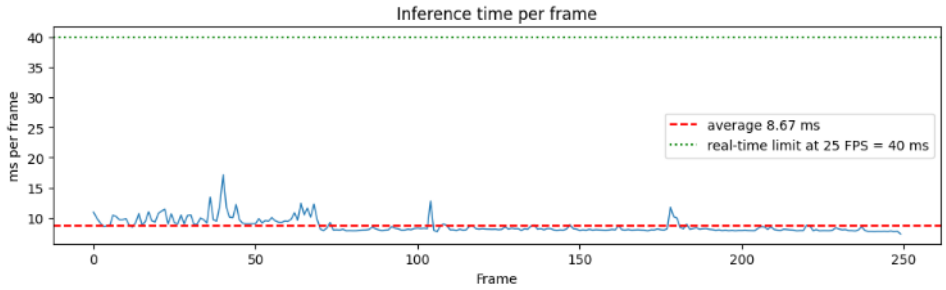


Figure 5. Inference time for every frame in Google Colab, with the 40 ms real-time budget for 25 FPS video.

In Google Colab the pipeline needed on average **8.67 ms per frame** (slowest frame 17.12 ms), a throughput of **115 FPS**. Even the slowest frame stays far below the 40 ms budget for 25 FPS video, so the pipeline runs in real time with a large safety margin, on a CPU and without any GPU.

Grayscale	Gaussian blur	Adaptive threshold	Morphological opening	Contours + filter	Classify + overlay	**Total**
0.22 ms (3%)	0.31 ms (4%)	6.11 ms (83%)	0.21 ms (3%)	0.20 ms (3%)	0.30 ms (4%)	**7.34 ms**

Table 4. Average time per pipeline stage (measured in a separate profiling run on a different CPU, so absolute values differ slightly from Colab; the shares are what matters).

The adaptive threshold dominates the cost, at about 83% of the total, because a Gaussian-weighted mean over a 101 x 101 window is computed for every pixel. Every other stage takes well under 1 ms.

5.3 Parameter sensitivity and noise robustness

To understand how robust the design is, each key parameter was varied while the others were kept at their defaults, and the video was re-generated with stronger camera noise. These runs were done outside Colab with the same code. The table shows the share of frames in which all class counts were exactly right (default values in bold).

Parameter	Value: frames with all classes exact
approxPolyDP epsilon	0.02: 21.6%, 0.03: 47.2%, 0.04: **97.2%**, 0.05: 13.2%, 0.06: 11.6%
Gaussian kernel size (1 = no blur)	1: 100.0%, 3: 97.2%, 5: 100.0%, 7: **97.2%**, 9: 97.2%, 11: 97.6%
Threshold block size	31: 16.4%, 51: 100.0%, 101: **97.2%**, 151: 99.2%
Camera noise std. dev.	6: 98.4%, 12: **97.2%**, 24: 95.2%, 36: 96.0%, 48: 93.2%

Table 5. Sensitivity of classification accuracy to the main parameters and to camera noise.

Three findings stand out. First, the shape tolerance epsilon is the most sensitive parameter: at 0.02 or 0.03 the slightly rounded tips of triangles survive as an extra vertex, so no triangle is ever found, while at 0.05 and above circles are simplified to only 4 or 5 vertices and are mistaken for squares; 0.04 is clearly the best setting. Second, the threshold block must be larger than the objects: with a 31-pixel block the window fits inside an object, its centre is no brighter than its surroundings, and most objects are lost; 51 or more works well. Third, detection is very robust to noise: the total count stays 100% correct even with four times the default noise (it only drops, to 16.8%, with the 31-pixel block), and classification degrades only gradually.

6. Error analysis and limitations

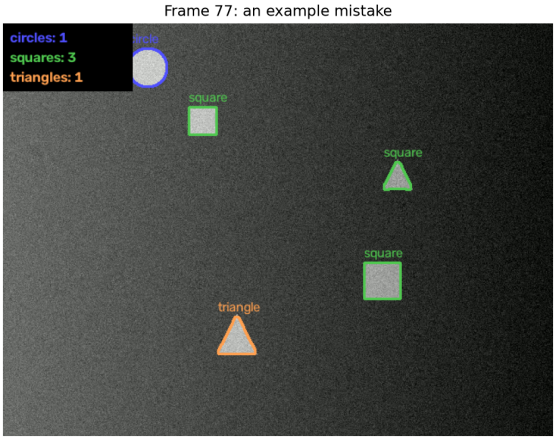


Figure 6. Frame 77: the small, dim triangle on the right is reported as a square.

All 7 imperfect frames (frames 77, 80, 104, 115, 117, 131, 170) contain the same single mistake: one triangle reported as a square. In each case it is the smallest triangle, in the darkest part of the frame. The Gaussian blur and the local threshold round off its sharp top corner, so the simplified outline keeps a short extra edge and four vertices. Table 5 supports this explanation: without blur (kernel size 1) or with a 51-pixel threshold block, all frames are correct.

Limitations of the current design:

- **Occlusion:** objects that touch or overlap form one blob and are counted as one object. In early tests with overlapping objects, the total count was correct in only about half of the frames.
- **Fixed rules:** classification relies on hand-tuned parameters and only handles simple geometric shapes. Real-world classes (people, cars) need a learned model.
- **Synthetic data:** the test video is controlled. Real footage adds motion blur, shadows, perspective and colour.

7. Future work

- Use a pre-trained deep learning detector such as **YOLO** for real object classes, keeping the adaptive-threshold pipeline for simple industrial scenes where it is far cheaper.
- Add **tracking** (centroid tracking or a Kalman filter) so objects keep their identity between frames and touching objects can be separated with watershed segmentation.
- Replace the Gaussian-weighted adaptive threshold with the mean-based variant, which in a quick timing test cut that stage from about 6 ms to under 1 ms; its accuracy would need to be re-validated.
- Tune epsilon, blur and block size automatically on a validation video, and use a 51-pixel block, which removed the remaining errors in Table 5.

8. Conclusion

The project delivered a complete, real-time computer vision pipeline in Python and OpenCV. Gaussian filtering and adaptive threshold matrices extract objects reliably despite strong lighting gradients and noise, where a global threshold fails (100% vs. 6.8% correct total counts). Contour-based shape classification is correct in 97.2% of frames, and the dynamic overlay shows the live count of every class. At 8.67 ms per frame (115 FPS) on a CPU, the system comfortably meets real-time requirements. The frame-by-frame evaluation also identified exactly where and why it fails, and which parameter changes would fix it.

Appendix: Source code (run in Google Colab)

```
# Task 4: Computer Vision Object Detector & Frame-by-Frame Segmenter Pipeline (code as run in Google Colab)

# ---- Step 1
import cv2, numpy as np, pandas as pd
import matplotlib.pyplot as plt

W, H, FPS, N_FRAMES = 640, 480, 25, 250           # 10-second video
rng = np.random.default_rng(42)
KINDS = ["circle", "square", "triangle"]

# 7 moving shapes, each in its own horizontal lane so they never overlap;
# each one appears and disappears at its own time
shapes = []
for i in range(7):
    start = int(rng.integers(0, 120))
    shapes.append({"kind": KINDS[i % 3], "size": int(rng.integers(16, 26)),
                   "x": float(rng.uniform(60, W - 60)), "lane": 50 + 63 * i, "y": 50 + 63 * i,
                   "vx": float(rng.choice([-1, 1]) * rng.uniform(2, 6)),
                   "start": start, "end": start + int(rng.integers(90, 200))})

def draw_shape(img, s):
    x, y, r = int(s["x"]), int(s["y"]), s["size"]
    if s["kind"] == "circle":
        cv2.circle(img, (x, y), r, 235, -1)
    elif s["kind"] == "square":
        cv2.rectangle(img, (x - r, y - r), (x + r, y + r), 235, -1)
    else:
        pts = np.array([[x, y - r], [x - r, y + r], [x + r, y + r]], np.int32)
        cv2.fillPoly(img, [pts], 235)

# uneven lighting: bright on the left, dark on the right (this is why we need ADAPTIVE threshold)
light = np.tile(np.linspace(110, 20, W), (H, 1))

writer = cv2.VideoWriter("shapes.mp4", cv2.VideoWriter_fourcc(*"mp4v"), FPS, (W, H))
truth = []                                        # ground truth: real count of each shape per frame
for f in range(N_FRAMES):
    frame = np.zeros((H, W), np.uint8)
    visible = [s for s in shapes if s["start"] <= f < s["end"]]
    for s in visible:
        draw_shape(frame, s)
    frame = frame.astype(float) * (0.45 + light / 200)          # shapes get darker on the right
    frame = np.clip(np.maximum(frame, light) + rng.normal(0, 12, (H, W)), 0, 255).astype(np.uint8)  # + camera noise
    writer.write(cv2.cvtColor(frame, cv2.COLOR_GRAY2BGR))
    truth.append({"frame": f, **{k: sum(s["kind"] == k for s in visible) for k in KINDS}})
    for s in shapes:                              # move sideways (with a small wobble) and bounce off the edges
        s["x"] += s["vx"]
        if not 45 < s["x"] < W - 45: s["vx"] *= -1
        s["y"] = s["lane"] + 6 * np.sin(f / 8 + s["lane"])
writer.release()

truth = pd.DataFrame(truth)
truth.to_csv("ground_truth.csv", index=False)
print("Video saved: shapes.mp4,", N_FRAMES, "frames")
print("Average shapes on screen:", truth[KINDS].sum(axis=1).mean().round(1))

cap = cv2.VideoCapture("shapes.mp4"); cap.set(cv2.CAP_PROP_POS_FRAMES, 120); ok, frame = cap.read(); cap.release()
plt.figure(figsize=(7, 5)); plt.imshow(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)); plt.title("Frame 120 of the test
      video"); plt.axis("off"); plt.show()

# ---- Step 2
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)                # 1. colour -> grey
blur = cv2.GaussianBlur(gray, (7, 7), 0)                       # 2. Gaussian filter removes grainy noise

# 3a. ONE threshold for the whole picture (Otsu picks it automatically)
_, global_mask = cv2.threshold(blur, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

# 3b. ADAPTIVE threshold: each pixel is compared with its own 101x101 neighbourhood;
#     it becomes white only if it is at least 15 brighter than its surroundings
mask = cv2.adaptiveThreshold(blur, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY, 101, -15)
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, np.ones((5, 5), np.uint8))   # 4. remove tiny white specks

fig, axes = plt.subplots(2, 2, figsize=(10, 8))
for ax, img, title in zip(axes.ravel(), [gray, blur, global_mask, mask],
                          ["1. Grayscale frame", "2. After Gaussian blur",
                           "3a. Global threshold (Otsu)", "3b. Adaptive threshold + cleanup"]):
    ax.imshow(img, cmap="gray"); ax.set_title(title); ax.axis("off")
plt.tight_layout(); plt.savefig("threshold_comparison.png", dpi=150); plt.show()

# ---- Step 3
COLORS = {"circle": (255, 80, 80), "square": (80, 200, 80), "triangle": (80, 160, 255)}   # BGR colours

def classify(contour):
    """Name a shape by counting the corners of its simplified outline."""
    perimeter = cv2.arcLength(contour, True)
    corners = cv2.approxPolyDP(contour, 0.04 * perimeter, True)
```

```
    if len(corners) == 3:
        return "triangle"
    if len(corners) == 4:
        return "square"
    return "circle"                     # many corners = round

def process_frame(frame):
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    blur = cv2.GaussianBlur(gray, (7, 7), 0)
    mask = cv2.adaptiveThreshold(blur, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY, 101, -15)
    mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, np.ones((5, 5), np.uint8))
    contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    contours = [c for c in contours if cv2.contourArea(c) > 400]          # ignore tiny blobs
    counts = {k: 0 for k in KINDS}
    out = frame.copy()
    for c in contours:
        kind = classify(c)
        counts[kind] += 1
        x, y, w, h = cv2.boundingRect(c)
        cv2.drawContours(out, [c], -1, COLORS[kind], 2)                   # outline the shape
        cv2.putText(out, kind, (x, y - 6), cv2.FONT_HERSHEY_SIMPLEX, 0.5, COLORS[kind], 1)
    # dynamic count overlay in the top-left corner
    cv2.rectangle(out, (0, 0), (150, 78), (0, 0, 0), -1)
    for i, k in enumerate(KINDS):
        cv2.putText(out, f"{k}s: {counts[k]}", (8, 22 + 23 * i), cv2.FONT_HERSHEY_SIMPLEX, 0.55, COLORS[k], 2)
    return out, counts

annotated, counts = process_frame(frame)
print("Detected:", counts)
print("Truth:   ", truth.loc[120, KINDS].to_dict())
plt.figure(figsize=(8, 6)); plt.imshow(cv2.cvtColor(annotated, cv2.COLOR_BGR2RGB)); plt.axis("off")
plt.title("Frame 120 with detections"); plt.savefig("annotated_frame.png", dpi=150, bbox_inches="tight"); plt.show()

# ---- Step 4
import time

cap = cv2.VideoCapture("shapes.mp4")
out_video = cv2.VideoWriter("shapes_annotated.mp4", cv2.VideoWriter_fourcc(*"mp4v"), FPS, (W, H))
rows = []
f = 0
while True:
    ok, frm = cap.read()
    if not ok:
        break
    t0 = time.perf_counter()
    annotated, counts = process_frame(frm)                    # the full pipeline on one frame
    ms = (time.perf_counter() - t0) * 1000
    out_video.write(annotated)

    # baseline for comparison: the same pipeline but with ONE global (Otsu) threshold
    g = cv2.GaussianBlur(cv2.cvtColor(frm, cv2.COLOR_BGR2GRAY), (7, 7), 0)
    _, gm = cv2.threshold(g, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
    gc, _ = cv2.findContours(gm, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    global_total = len([c for c in gc if cv2.contourArea(c) > 400])

    t = truth.loc[f]
    rows.append({"frame": f, "ms": ms, **{f"pred_{k}": counts[k] for k in KINDS},
                 **{f"true_{k}": int(t[k]) for k in KINDS}, "global_total": global_total})
    f += 1
cap.release(); out_video.release()

m = pd.DataFrame(rows)
m["pred_total"] = m[[f"pred_{k}" for k in KINDS]].sum(axis=1)
m["true_total"] = m[[f"true_{k}" for k in KINDS]].sum(axis=1)
m.to_csv("frame_metrics.csv", index=False)                    # per-frame inference log

print(f"Frames processed: {len(m)}")
print(f"Average time per frame: {m.ms.mean():.2f} ms  ->  {1000 / m.ms.mean():.0f} FPS  (slowest frame {m.ms.max():.
      2f} ms)")
for k in KINDS:
    print(f"{k:9s} correct count in {100 * (m[f'pred_{k}'] == m[f'true_{k}']).mean():5.1f}% of frames | "
          f"mean abs. error {(m[f'pred_{k}'] - m[f'true_{k}']).abs().mean():.3f}")
print(f"TOTAL objects correct in {100 * (m.pred_total == m.true_total).mean():.1f}% of frames (adaptive)")
print(f"TOTAL objects correct in {100 * (m.global_total == m.true_total).mean():.1f}% of frames (global Otsu
      baseline)")
all_right = (m[[f"pred_{k}" for k in KINDS]].values == m[[f"true_{k}" for k in KINDS]].values).all(axis=1)
print(f"Every class exactly right in {100 * all_right.mean():.1f}% of frames")

# ---- Step 5
# Chart 1: processing time of every frame
fig, ax = plt.subplots(figsize=(10, 3.2))
ax.plot(m.frame, m.ms, lw=1, color="#1f77b4")
ax.axhline(m.ms.mean(), color="red", ls="--", label=f"average {m.ms.mean():.2f} ms")
ax.axhline(1000 / FPS, color="green", ls=":", label=f"real-time limit at {FPS} FPS = {1000 / FPS:.0f} ms")
ax.set_xlabel("Frame"); ax.set_ylabel("ms per frame"); ax.set_title("Inference time per frame"); ax.legend()
plt.tight_layout(); plt.savefig("latency_per_frame.png", dpi=150); plt.show()

# Chart 2: objects counted vs. real number, over the whole video
```

```
fig, ax = plt.subplots(figsize=(10, 3.2))
ax.step(m.frame, m.true_total, where="mid", lw=3, color="#cccccc", label="Ground truth")
ax.step(m.frame, m.pred_total, where="mid", lw=1.2, color="#1f77b4", label="Adaptive pipeline")
ax.step(m.frame, m.global_total, where="mid", lw=1, color="#d62728", alpha=0.7, label="Global threshold")
ax.set_xlabel("Frame"); ax.set_ylabel("Objects counted"); ax.set_title("Object count per frame"); ax.legend()
plt.tight_layout(); plt.savefig("count_over_time.png", dpi=150); plt.show()

# Chart 3: which frames had a wrong label, and what went wrong
wrong = m[~all_right]
print("Frames with a wrong label:", wrong.frame.tolist())
print(wrong[["frame"] + [f"pred_{k}" for k in KINDS] + [f"true_{k}" for k in KINDS]].head(10).to_string(index=False))

cap = cv2.VideoCapture("shapes.mp4"); cap.set(cv2.CAP_PROP_POS_FRAMES, int(wrong.frame.iloc[0])); ok, bad = cap.read(
      ); cap.release()
plt.figure(figsize=(8, 6)); plt.imshow(cv2.cvtColor(process_frame(bad)[0], cv2.COLOR_BGR2RGB)); plt.axis("off")
plt.title(f"Frame {int(wrong.frame.iloc[0])}: an example mistake"); plt.savefig("error_example.png", dpi=150,
      bbox_inches="tight"); plt.show()
```